Sistema de Préstamos Interbibliotecarios

TP Integrador de Desarrollo de Aplicaciones con Objetos (UTN FRC, 2026), Tema 1.

Aplicación de escritorio para una red de bibliotecas: sedes, socios, catálogo y ejemplares, préstamos locales e interbibliotecarios, envíos entre sedes con remito y trazabilidad, reservas, devoluciones con sanciones y reportes.

Python + SQLite (`sqlite3`, sin ORM) + Flet 1.x para la interfaz + ReportLab y Matplotlib para los reportes.

La aplicación tiene dos modos, que comparten los mismos servicios, modelo y base:

- **Personal de la biblioteca:** ingresa con una clave y administra sedes, socios, catálogo, préstamos, remitos, reservas y reportes.
- **Consulta para socios:** el socio ingresa con su DNI y consulta el catálogo y la disponibilidad por sede, sus préstamos (abiertos e historial), sus reservas y su situación (habilitado, suspendido o inhabilitado, multas impagas y préstamos abiertos sobre el máximo). Puede crear reservas desde el catálogo y cancelar las suyas; no ve datos de otros socios ni registra préstamos, devoluciones o remitos.

Puesta en marcha

```
python -m venv .venv
# Windows: .\.venv\Scripts\Activate.ps1    ·    Linux/macOS: source
.venv/bin/activate
python -m pip install -r requirements.txt

python -m biblioteca.datos.seed --reiniciar   # crea biblioteca.db con
datos de prueba
python main.py                                # abre la pantalla de ingreso
python -m pytest                              # 82 pruebas
```

Sin el seed, `main.py` crea una base vacía. Los reportes exportados se guardan en `exportes/`.

`main.py` abre una pantalla de ingreso con dos opciones: **Personal de la biblioteca** (clave `PERSONAL123` y "Ingresar") y **Socios** (DNI y "Consultar"). Cada modo tiene un botón "Salir" que vuelve a esa pantalla. DNI del seed para probar el modo socio:

DNI	Socio	Situación
33100200	Lucía Fernández	habilitada, con un préstamo activo
35210300	Martín Gómez	con un préstamo vencido
30987654	Valentina Ruiz	suspendida
36543210	Tomás Herrera	inhabilitado manualmente
40111222	Camila Torres	con una multa impaga y una reserva asignada

Limitaciones del ingreso (ninguno de los dos modos es una autenticación real):

- El personal ingresa con una única clave compartida, fija y escrita en el código. No hay usuarios, roles ni forma de cambiarla desde la aplicación.
- El socio ingresa solo con su DNI, sin contraseña. Es un modo de consulta: cualquiera que conozca un DNI puede ver los datos de ese socio y gestionar sus reservas.

Estructura

```
main.py                    ft.run(main)
biblioteca/
├── modelo/                Entidades y patrones. Python puro.
│   ├── remito.py          State (ciclo de vida) + Observer (sujeto)
│   ├── sancion.py         Strategy (multa o suspensión)
│   └── sede, socio, catalogo, prestamo, reserva, excepciones
├── datos/                 schema.sql, conexion.py, un DAO por entidad,
seed.py
├── servicios/             Casos de uso, validaciones y transacciones
│   ├── observadores.py    Observadores concretos del remito
│   ├── operaciones.py     Empaquetado en remitos y asignación de reservas
│   ├── consulta_socio.py  ServicioConsultaSocio: lo que el socio consulta
y gestiona
│   └── reportes.py        ServicioReportes: 6 reportes con SQL
├── reportes/              Gráficos (Matplotlib) y exportadores PDF
(ReportLab) y CSV
└── presentacion/          Flet: una clase por pantalla con build()
    ├── ingreso.py         Pantalla de ingreso (personal o socio por DNI)
    ├── app_base.py        AplicacionBase: navegación, mensajes y diálogos
comunes
    ├── app.py             AplicacionBiblioteca (personal) y main()
    ├── app_socio.py       AplicacionSocio (consulta para socios)
    ├── pantallas/         Pantallas del personal
    └── pantallas_socio/   Pantallas del socio
tests/                     Modelo, restricciones del esquema, servicios,
reportes e interfaz
                           (utilidades.py: página falsa para probar la
interfaz sin ventanas)
```

Regla de capas: la interfaz solo llama a los servicios; los servicios usan el modelo y los DAO; el modelo no conoce la base. Cada servicio abre una conexión, se la pasa a los DAO y hace un único `commit()` o `rollback()`.

Patrones de diseño

Patrón	Dónde	Qué resuelve
State	`modelo/remito.py`	Cada estado del remito (`EnPreparacion`, `Despachado`, `EnTransito`, `Recibido`, `RecibidoConIncidencia`, `Extraviado`, `Anulado`) decide qué operaciones admite; las demás lanzan

Patrón	Dónde	Qué resuelve
		`TransicionInvalida`. Cada transición queda en el historial (`remito_movimiento`).
Strategy	`modelo/sancion.py`	`SancionMonetaria` o `SancionSuspension`. Cada sede configura su política; al devolver con atraso se aplica la de la **sede de origen** del préstamo (`Sede.estrategia_sancion()` → `CalculadoraSancion`).
Observer	`modelo/remito.py` + `servicios/observadores.py`	El remito notifica cada cambio de estado; `ActualizadorEjemplares` mueve y actualiza los ejemplares, `ActualizadorPrestamos` avanza o cancela los préstamos. El remito no sabe qué hacen.
DAO	`datos/*_dao.py`	Todo el SQL encapsulado, un DAO por entidad, mapeando filas a objetos.

Reglas de negocio definidas por el equipo

- Plazos: préstamo local 14 días; interbibliotecario 21 días desde que el socio lo retira.
- Máximo 3 préstamos abiertos por socio.
- Un socio está inhabilitado si tiene una suspensión vigente o una inhabilitación manual (con motivo).
- Solo se prestan ejemplares disponibles, que pertenezcan a la sede de origen y estén físicamente en ella.
- No se inician préstamos, reservas ni envíos hacia o desde una sede dada de baja, y no se da de baja una sede con envíos en curso.
- Préstamo interbibliotecario: el ejemplar se aparta y se agrega al remito en preparación entre las dos sedes (o se crea uno).
- Recepción: correcto o dañado → el préstamo queda listo para retirar; faltante o destruido → se cancela, y el ejemplar queda extraviado o de baja.
- Si un ejemplar se devuelve en una sede que no es la suya, se genera solo un remito de retorno.
- Reservas: por título, solo en sedes que tienen ejemplares de ese título. Cuando se libera un ejemplar se aparta para la reserva más antigua y el socio tiene 3 días para retirarlo; si no, la reserva expira.
- Al abrir la aplicación se marcan los préstamos vencidos y expiran las reservas no retiradas.

Reportes

En pantalla y exportables a PDF (con gráfico) y CSV:

1. Préstamos abiertos y material en tránsito
2. Préstamos vencidos y sanciones por socio
3. Libros más solicitados entre sedes
4. Disponibilidad del catálogo por sede
5. Tasa de envíos exitosos e incidencias
6. Tiempo promedio de tránsito entre sedes (desde el historial del remito)

Datos de prueba

`seed.py` carga 5 sedes (una dada de baja), 12 socios, 16 libros y 34 ejemplares, y simula 70 días de actividad **a través de los servicios**, con un reloj que avanza: préstamos en término y con atraso, multas (una pagada), suspensiones, remitos recibidos, con daño, con faltante, extraviado y anulado, reservas cumplidas, canceladas, asignadas y pendientes. Termina el día anterior a la carga, así que siempre hay préstamos vencidos y envíos en curso relativos a "hoy".